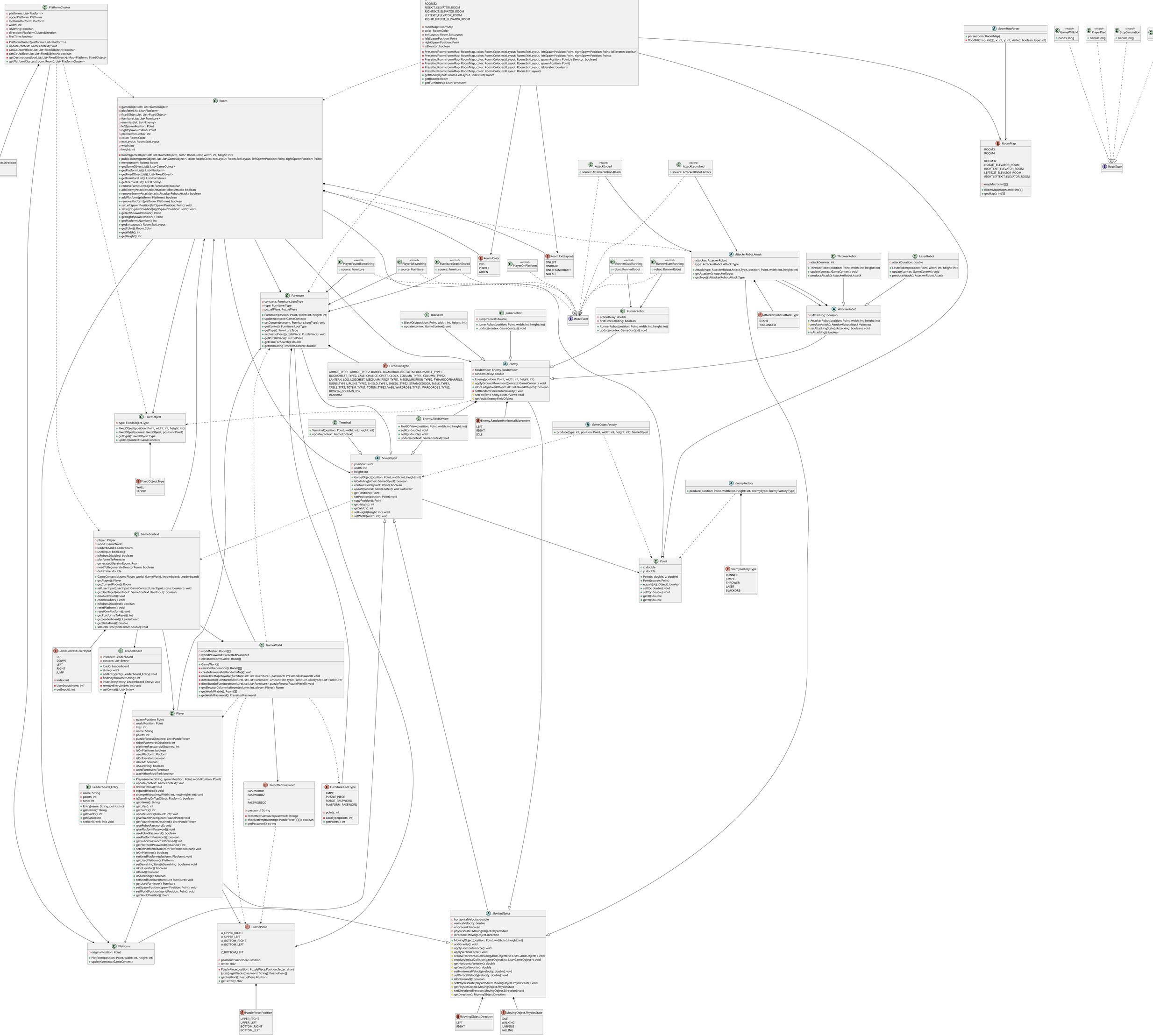
PlatformCluster
- platforms: List<Platform>
- upperPlatform: Platform
- lbottomPlatform: Platform
- width: int
- isMoving: boolean
- direction: PlatformCluster.Direction
- firstTime: boolean
+ PlatformCluster(platforms: List<Platform>)
+ update(context: GameContext): void
- canGoDown(floorList: List<FixedObject>): boolean
- canGoUp(floorList: List<FixedObject>): boolean
+ getDestinations(sortList: List<FixedObject>): Map<Platform, FixedObject>
+ getPlatformClusters(room: Room): List<PlatformCluster>
PlatformCluster.Direction
UP
DOWN
PresettedRoom
ROOM3
ROOM4
...
ROOM32
NOEXIT_ELEVATOR_ROOM
RIGHTEXIT_ELEVATOR_ROOM
LEFTEXIT_ELEVATOR_ROOM
RIGHTLEFTEXIT_ELEVATOR_ROOM
- roomMap: RoomMap
- color: Room.Color
- exitLayout: Room.ExitLayout
- leftSpawnPosition: Point
- rightSpawnPosition: Point
- isElevator: boolean
+ PresettedRoom(roomMap: RoomMap, color: Room.Color, exitLayout: Room.ExitLayout, leftSpawnPosition: Point, rightSpawnPosition: Point, isElevator: boolean)
+ PresettedRoom(roomMap: RoomMap, color: Room.Color, exitLayout: Room.ExitLayout, leftSpawnPosition: Point, rightSpawnPosition: Point)
+ PresettedRoom(roomMap: RoomMap, color: Room.Color, exitLayout: Room.ExitLayout, spawnPosition: Point, isElevator: boolean)
+ PresettedRoom(roomMap: RoomMap, color: Room.Color, exitLayout: Room.ExitLayout, spawnPosition: Point)
+ PresettedRoom(roomMap: RoomMap, color: Room.Color, exitLayout: Room.ExitLayout, isElevator: boolean)
+ PresettedRoom(roomMap: RoomMap, color: Room.Color, exitLayout: Room.ExitLayout)
+ getRoom(layout: Room.ExitLayout, index: int): Room
+ getRoom(): Room
+ getFurnitures(): List<Furniture>
RoomMapParser
+ parse(room: RoomMap)
- floodFill(map: int[][], x: int, y: int, visited: boolean, type: int)
«record»
GameWillEnd
+ nanos: long
«record»
PlayerDied
+ nanos: long
«record»
StopSimulation
+ nanos: long
«record»
TerminalOpened
ModelState
Room
- gameObjectList: List<GameObject>
- platformList: List<Platform>
- fixedObjectList: List<FixedObject>
- furnitureList: List<Furniture>
- enemiesList: List<Enemy>
- leftSpawnPosition: Point
- rightSpawnPosition: Point
- platformsNumber: int
- color: Room.Color
- exitLayout: Room.ExitLayout
- width: int
- height: int
+ Room(gameObjectList: List<GameObject>, color: Room.Color, width: int, height: int)
+ public Room(gameObjectList: List<GameObject>, color: Room.Color, exitLayout: Room.ExitLayout, leftSpawnPosition: Point, rightSpawnPosition: Point)
+ merge(room: Room): Room
+ getGameObjectList(): List<GameObject>
+ getPlatformList(): List<Platform>
+ getFixedObjectList(): List<FixedObject>
+ getFurnitureList(): List<Furniture>
+ getEnemiesList(): List<Enemy>
+ removeFurniture(object: Furniture): boolean
+ addEnemyAttack(attack: AttackerRobot.Attack): boolean
+ removeEnemyAttack(attack: AttackerRobot.Attack): boolean
+ addPlatform(platform: Platform): boolean
+ removePlatform(platform: Platform): boolean
+ setLeftSpawnPosition(leftSpawnPosition: Point): void
+ setRightSpawnPosition(rightSpawnPosition: Point): void
+ getLeftSpawnPosition(): Point
+ getRightSpawnPosition(): Point
+ getPlatformsNumber(): int
+ getExitLayout(): Room.ExitLayout
+ getColor(): Room.Color
+ getWidth(): int
+ getHeight(): int
RoomMap
ROOM3
ROOM4
...
ROOM32
NOEXIT_ELEVATOR_ROOM
RIGHTEXIT_ELEVATOR_ROOM
LEFTEXIT_ELEVATOR_ROOM
RIGHTLEFTEXIT_ELEVATOR_ROOM
- mapMatrix: int[][]
+ RoomMap(mapMatrix: int[][])
+ getMap(): int[][]
«record»
AttackEnded
+ source: AttackerRobot.Attack
«record»
AttackLaunched
+ source: AttackerRobot.Attack
«record»
PlayerFoundSomething
+ source: Furniture
«record»
PlayerIsSearching
+ source: Furniture
«record»
FurnitureSearchEnded
+ source: Furniture
Room.Color
RED
PURPLE
GREEN
«record»
PlayerOnPlatform
Room.ExitLayout
ONLEFT
ONRIGHT
ONLEFTANDRIGHT
NOEXIT
AttackerRobot.Attack
- attacker: AttackerRobot
- type: AttackerRobot.Attack.Type
+ Attack(type: AttackerRobot.Attack.Type, position: Point, width: int, height: int)
+ getAttacker(): AttackerRobot
+ getType(): AttackerRobot.Attack.Type
«record»
RunnerStopRunning
+ robot: RunnerRobot
«record»
RunnerStartRunning
+ robot: RunnerRobot
ThrowerRobot
- attackCounter: int
+ ThrowerRobot(position: Point, width: int, height: int)
+ update(contex: GameContext): void
+ produceAttack(): AttackerRobot.Attack
LaserRobot
- attackDuration: double
+ LaserRobot(position: Point, width: int, height: int)
+ update(contex: GameContext): void
+ produceAttack(): AttackerRobot.Attack
Furniture
- contexte: Furniture.LootType
- type: Furniture.Type
- puzzelPiece: PuzzlePiece
+ Furniture(position: Point, width: int, height: int)
+ update(context: GameContext)
+ setContent(content: Furniture.LootType): void
+ getContet(): Furniture.LootType
+ getType(): Furniture.Type
+ setPuzzlePiece(puzzlePiece: PuzzlePiece): void
+ getPuzzlePiece(): PuzzlePiece
+ getTimeForSearch(): double
+ getRemainingTimeForSearch(): double
BlackOrb
+ BlackOrb(position: Point, width: int, height: int)
+ update(contex: GameContext): void
JumperRobot
- jumpInterval: double
+ JumperRobot(position: Point, width: int, height: int)
+ update(contex: GameContext): void
ModelEvent
RunnerRobot
- actionDelay: double
- firstTimeColliding: boolean
+ RunnerRobot(position: Point, width: int, height: int)
+ update(contex: GameContext): void
AttackerRobot.Attack.Type
ISTANT
PROLONGED
AttackerRobot
- isAttacking: boolean
+ AttackerRobot(position: Point, width: int, height: int)
# produceAttack(): AttackerRobot.Attack //abstract
+ setAttackingState(isAttacking: boolean): void
+ isAttacking(): boolean
Furniture.Type
ARMOR_TYPE1, ARMOR_TYPE2, BARREL, BIGMIRROR, BIGTOTEM, BOOKSHELF_TYPE1, BOOKSHELFT_TYPE2, CAVE, CHALICE, CHEST, CLOCK, COLUMN_TYPE1, COLUMN_TYPE2, LANTERN, LOG, LOGCHEST, MEDIUMMIRROR_TYPE1, MEDIUMMIRROR_TYPE2, PYRAMIDOFBARRELS, RUINS_TYPE1, RUINS_TYPE2, SHIELD_TYPE1, SHIELI_TYPE2, STRANGEDOOR, TABLE_TYPE1, TABLE_TYPE2, TOTEM_TYPE1, TOTEM_TYPE2, VASE, WARDROBE_TYPE1, WARDOROBE_TYPE2, BROKEN_COLUMN, IDK, RANDOM
Enemy
- fieldOfView: Enemy.FieldOfView
- randomDelay: double
+ Enemy(position: Point, width: int, height: int)
# applyGroundMovement(context: GameContext): void
+ isOnLedge(fixedObjectList: List<FixedObject>): boolean
+ setRandomHorizontalVelocity(): void
+ setFovfov: Enemy.FieldOfView): void
+ getFov(): Enemy.FieldOfView
FixedObject
- type: FixedObject.Type
+ FixedObject(position: Point, width: int, height: int)
+ FixedObject(source: FixedObject, position: Point)
+ getType(): FixedObject.Type
+ update(context: GameContext)
Terminal
+ Terminal(position: Point, width: int, height: int)
+ update(context: GameContext)
Enemy.FieldOfView
+ FieldOfView(position: Point, width: int, height: int)
+ setX(x: double): void
+ setY(y: double): void
+ update(context: GameContext): void
Enemy.RandomHorizontallMovement
LEFT
RIGHT
IDLE
GameObjectFactory
+ produce(type: int, position: Point, width: int, height: int): GameObject
GameObject
- position: Point
- width: int
- height: int
+ GameObject(position: Point, width: int, height: int)
+ isColliding(other: GameObject): boolean
+ containsPoint(point: Point): boolean
# update(context: GameContext): void //abstract
# getPosition(): Point
# setPosition(position: Point): void
+ copyPosition(): Point
+ getHeight(): int
+ getWidth(): int
# setHeight(height: int): void
# setWidth(width: int): void
EnemyFactory
+ produce(position: Point, width: int, height: int, enemyType: EnemyFactory.Type)
FixedObject.Type
WALL
FLOOR
GameContext
- player: Player
- world: GameWorld
- leaderboard: Leaderboard
- userInput: boolean[]
- isRobotsDisabled: boolean
- platformsToReset: in
- generatedElevatorRoom: Room
- needToRegenerateElevatorRoom: boolean
- deltaTime: double
+ GameContext(player: Player, world: GameWorld, leaderboard: Leaderboard)
+ getPlayer(): Player
+ getCurrentRoom(): Room
+ setUserInput(userInput: GameContext.UserInput, state: boolean): void
+ getUserInput(userInput: GameContext.UserInput): boolean
+ disableRobots(): void
+ enableRobots(): void
+ isRobotsDisabled(): boolean
+ resetPlatform(): void
+ resetOnePlatform(): void
+ getPlatformsToReset(): int
+ getLeaderboard(): Leaderboard
+ getDeltaTime(): double
+ setDeltaTime(deltaTime: double): void
Point
+ x: double
+ y: double
+ Point(x: double, y: double)
+ Point(source: Point)
+ equals(obj: Object): boolean
+ setX(x: double): void
+ setY(y: double): void
+ getX(): double
+ getY(): double
EnemyFactory.Type
RUNNER
JUMPER
THROWER
LASER
BLACKORB
GameContext.UserInput
UP
DOWN
LEFT
RIGHT
JUMP
- index: int
- UserInput(index: int)
+ getInput(): int
Leaderboard
- instance: Leaderboard
- content: List<Entry>
+ load(): Leaderboard
+ store(): void
+ add(Entry(entry Leaderboard_Entry): void
- findPlayer(name: String): int
- insertEntry(entry: Leaderboard_Entry): void
- removeEntry(index: int): void
+ getContet(): List<Entry>
GameWorld
- worldMatrix: Room[][]
- worldPassword: PresettedPassword
- elevatorRoomsCache: Room[]
+ GameWorld()
- randomGeneration(): Room[][]
- createTraversableRandomMap(): void
- makeTheMapPlayable(furnitureList: List<Furniture>, password: PresettedPassword): void
- distributeInFurniture(furnitureList: List<Furniture>, amount: int, type: Furniture.LootType): List<Furniture>
- distributeInFurniture(furnitureList: List<Furniture>, puzzlePieces: PuzzlePiece[]): void
+ getElevatorColumnAsRoom(column: int, player: Player): Room
+ getWorldMatrix(): Room[][]
+ getWorldPassword(): PresettedPassword
Player
- spawnPosition: Point
- worldPosition: Point
- lifes: int
- name: String
- points: int
- puzzlePiecesObtained: List<PuzzlePiece>
- robotPasswordsObtained: int
- platformPasswordsObtained: int
- isOnPlatform: boolean
- usedPlatform: Platform
- isOnElevator: boolean
- isDead: boolean
- isSearching: boolean
- usedFurniture: Furniture
- wasHitboxModified: boolean
+ Player(name: String, spawnPosition: Point, worldPosition: Point)
+ update(context: GameContext): void
- shrinkHitbox(): void
- expandHitbox(): void
- changeHitbox(newWidth: int, newHeight: int): void
+ isStandingOnTopOf(obj: Platform): boolean
+ getName(): String
+ getLifes(): int
+ getPoints(): int
+ updatePoints(amount: int): void
+ givePuzzlePiece(piece: PuzzlePiece): void
+ getPuzzlePiecesObtained(): List<PuzzlePiece>
+ giveRobotPassword(): void
+ givePlatformPassword(): void
+ useRobotPassword(): boolean
+ usePlatformPassword(): boolean
+ getRobotPasswordsObtained(): int
+ getPlatformPasswordsObtained(): int
+ setOnPlatformState(isOnPlatform: boolean): void
+ isOnPlatform(): boolean
+ setUsedPlatform(platform: Platform): void
+ getUsedPlatform(): Platform
+ setSearchingState(isSearching: boolean): void
+ isOnElevator(): boolean
+ isDead(): boolean
+ isSearching(): boolean
+ setUsedFurniture(furniture Furniture): void
+ getUsedFurniture(): Furniture
+ setSpawnPosition(spawnPosition: Point): void
+ setWorldPosition(worldPosition: Point): void
+ getWorldPosition(): Point
Leaderboard_Entry
- name: String
- points: int
- rank: int
+ Entry(name: String, points: int)
+ getName(): String
+ getPoints(): int
+ getRank(): int
+ setRank(rank: int): void
PresettedPassword
PASSWORD1
PASSWORD2
...
PASSWORD20
- password: String
- PresettedPassword(password: String)
+ checkAttempt(attempt: PuzzlePiece[][][]): boolean
+ getPassword(): string
Furniture.LootType
EMPY
PUZZLE_PIECE
ROBOT_PASSWORD
PLATFORM_PASSWORD
- points: int
- LootType(points: int)
+ getPoints(): int
PuzzlePiece
A_UPPER_RIGHT
A_UPPER_LEFT
A_BOTTOM_RIGHT
A_BOTTOM_LEFT
...
Z_BOTTOM_LEFT
- position: PuzzlePiece.Position
- letter: char
- PuzzlePiece(position: PuzzlePiece.Position, letter: char)
- {static}+getPieces(password: String): PuzzlePiece[]
+ getPosition(): PuzzlePiece.Position
+ getLetter(): char
Platform
- originalPosition: Point
+ Platform(position: Point, width: int, height: int)
+ update(context: GameContext)
MovingObject
- horizontalVelocity: double
- verticalVelocity: double
- onGround: boolean
- physicsState: MovingObject.PhysicsState
- direction: MovingObject.Direction
+ MovingObject(position: Point, width: int, height: int)
# addGravity(): void
# applyHorizontalForce(): void
# applyVerticalForce(): void
# resolveHorizontalCollision(gameObjectList: List<GameObject>): void
# resolveVerticalCollision(gameObjectList: List<GameObject>): void
# getHorizontalVelocity(): double
# getVerticalVelocity(): double
# setHorizontalVelocity(velocity: double): void
# setVerticalVelocity(velocity: double): void
+ isOnGround(): boolean
# setPhysicsState(physicsState: MovingObject.PhysicsState): void
# getPhysicsState(): MovingObject.PhysicsState
# setDirection(direction: MovingObject.Direction): void
# getDirection(): MovingObject.Direction
PuzzlePiece.Position
UPPER_RIGHT
UPPER_LEFT
BOTTOM_RIGHT
BOTTOM_LEFT
MovingObject.Direction
LEFT
RIGHT
MovingObject.PhysicsState
IDLE
WALKING
JUMPING
FALLING